Q4: Google AI Artifact — Materealized Web (Concept & Technical Plan)

The Question

> Use one of Google's AI tools to create a digital artifact to showcase who you are and what you love to do, creatively. Add a link to what you've created here. (750 characters max)

The Concept

Materealized Web — a browser-based version of the Materealized thesis installation. Visitors describe portions of their remembered home, and the system generates particle-cloud visualizations of those memories in real time, assembling them into a navigable grid that reconstructs a fragmented, dreamy version of a place that may no longer exist.

User Experience

1. Land on a minimal dark page
2. One prompt: "What do you remember about your home?"
3. User types or speaks a description of one part of their home
4. Gemini generates an image of that described space
5. Image is converted to a point-cloud / gaussian-splat style particle rendering
6. The rendering appears in the center cell of a spatial grid
7. User scrolls/clicks to adjacent cell, describes the next room/space

8. Over time, they build a fragmented reconstruction of a remembered place

Interaction Design

- "I remember..." triggers listening/input
- Voice or text input
- Generation happens automatically after input
- Scroll wheel moves to adjacent grid cell
- Can redo a cell by staying on it and describing again
- Simple, intuitive, but doing something technically complex underneath

---

Technical Architecture

Pipeline

```
Voice/Text Input → Web Speech API → Prompt Construction → Gem
ini API (image gen) → Generated Image → Depth Estimation → Th
ree.js Point Cloud Rendering → Grid Cell Placement
```

Google AI Tools Used

- **Gemini API** via Google AI Studio — image generation using `gemini-2.5-flash-image` (Nano Banana)
- **Google AI Studio** — API key, prompt prototyping
- Optional: **Gemini Vision** for depth estimation from generated image

Front-End Stack

- **Three.js** — particle/point-cloud rendering, camera controls, grid system
- **Web Speech API** — browser-native voice input
- **Vanilla JS** — lightweight, no framework needed
- **Static hosting** — GitHub Pages, Vercel, or Netlify

Depth-to-Particles Pipeline

1. Gemini generates image from memory description
2. Image loaded into canvas
3. Depth estimation applied (options below)
4. For each pixel: x/y from position, z from depth value, color from image pixel
5. Three.js creates particle buffer geometry from this data
6. Particles rendered with appropriate size, opacity, and color
7. Camera orbits subtly for that dreamy floating feel

Depth Estimation Options

- **Option A: Gemini Vision** — Send generated image back to Gemini with prompt "Estimate depth map for this image, return as grayscale" (uncertain if this works well enough)
- **Option B: Luminance-based hack** — Use brightness as a proxy for depth (simpler, less accurate, but fast)
- **Option C: TensorFlow.js + MiDaS** — Client-side ML depth estimation (most accurate, heavier)
- **Option D: Pre-prompted depth** — Ask Gemini to generate the image WITH depth information baked into the prompt (e.g., strong foreground/background separation)

Grid System

- 2D grid (e.g., 5×5 or 3×3)
- Each cell holds one generated memory
- Scroll wheel or arrow keys to navigate
- Camera smoothly transitions between cells
- Empty cells show wireframe or subtle indicator
- Filled cells show particle cloud of that memory

Visual Reference

- https://particles.casberry.in/ — particle displacement from image
- Gaussian splat aesthetic
- Dreamy, floating, fragmented feel
- Dark background, bright colored particles

Application Answer (750 chars max)

Draft — to be written after artifact is built

Should describe: what it is, what Google AI tool was used, why it matters to you, and link.

Timeline Estimate

- **4-6 hours minimum** for a working MVP
- **Core features:** single cell generation + particle rendering
- **Stretch features:** grid navigation, voice input, multiple cells, ambient audio
- **Polish features:** loading animations, subtle camera drift, responsive design

---

FULL GOOGLE-ONLY STACK (Final Plan)

Why Google-Only Matters

The application says "Use one of Google's AI tools." But Gabe wants to go further: use ONLY Google tools for the entire stack. This demonstrates:

- Comfort with Google Cloud ecosystem (listed as "major plus" in CT role)
- TypeScript (listed as front-end requirement — write the project in TS)
- Python or Node.js (listed as back-end requirement — Cloud Functions)
- AI technologies (Gemini API)

- Experimental UI + interfaces (the particle/point-cloud interaction)

Complete Google Tool Stack

Layer	Google Tool	Purpose
AI Image Generation	**Gemini API** (`gemini-2.5-flash-image` / Nano Banana) via Google AI Studio	Generate images from memory descriptions
Depth Estimation	**Gemini Vision** (send generated image back to Gemini, ask for depth map)	Create depth data for particle displacement
API Key Security	**Firebase Cloud Functions** (Node.js/TypeScript)	Proxy Gemini API calls, keep API key server-side
Hosting	**Firebase Hosting** (free tier: 10GB storage, 360MB/day transfer)	Host the static front-end on Google CDN
Front-End Language	**TypeScript** (compiled to JS)	Demonstrates the front-end skill the CT role requires
3D Rendering	**Three.js** (open source, loaded via CDN)	Point cloud / particle rendering
Voice Input	**Web Speech API** (Chrome-native, built by Google)	Speech-to-text in the browser
Optional: Analytics	**Google Analytics** or **Firebase Analytics**	Track usage if desired
Optional: Auth	**Firebase Auth** (anonymous)	Rate limit per user if needed

What This Project Demonstrates (mapped to role requirements)

CT Role: "solid foundation in front-end development (TypeScript)"

→ Entire front-end written in TypeScript

CT Role: "back-end development (Python, node.js, Go)"

→ Firebase Cloud Function written in Node.js/TypeScript to proxy Gemini API

CT Role: "AI technologies"

→ Gemini API for image generation + vision for depth estimation

CT Role: "server-side experience with Google Cloud"

→ Firebase Hosting + Cloud Functions = Google Cloud infrastructure

CT Role: "hardware, interfaces, experimental UI"

→ Voice input + 3D particle rendering + spatial grid navigation = experimental interface

CT Role: "prototype interactive demos"

→ This IS an interactive demo

STEP-BY-STEP BUILD PLAN

Phase 1: Setup (1-2 hours)

1. Create a Google AI Studio account → get free Gemini API key
2. Create a Firebase project → enable Hosting + Cloud Functions
3. Initialize local project with `firebase init` (select Hosting + Functions)
4. Set up TypeScript compilation for front-end
5. Test Gemini API call from Cloud Function

Phase 2: Core Generation Pipeline (3-4 hours)

1. Build the Cloud Function that:
    - Receives a text prompt from the front-end
    - Calls Gemini API to generate an image
    - Optionally calls Gemini Vision to estimate depth from the image
    - Returns base64 image + depth data to the front-end
2. Test end-to-end: text in → image out

Phase 3: Particle Rendering (3-4 hours)

1. Set up Three.js scene with dark background
2. Load generated image into canvas
3. Extract pixel data (color + position)
4. Apply depth displacement to create 3D point cloud
5. Render particles with appropriate size, opacity, color
6. Add subtle camera orbit/drift for dreamy effect
7. Test with a single generated image

Phase 4: Interaction Design (2-3 hours)

1. Build the prompt UI: "What do you remember about your home?"
2. Add text input (and optionally voice via Web Speech API)
3. Connect input → Cloud Function → generation → particle rendering
4. Add loading state / generation animation
5. Handle the "I remember..." trigger

Phase 5: Grid Navigation (2-3 hours)

1. Build spatial grid system (start with 3×3)
2. Place generated particle cloud in center cell
3. Add scroll/arrow key navigation to adjacent cells
4. Smooth camera transition between cells
5. Show wireframe/indicator for empty cells
6. Allow re-generation on current cell

Phase 6: Polish + Deploy (2-3 hours)

1. Add ambient visual effects (subtle particle drift, glow)
2. Add loading animations during generation

3. Responsive design check
4. Deploy to Firebase Hosting
5. Test live URL
6. Write the 750-character application answer

Total Estimated Time: 13-19 hours across 3+ days

MVP vs. STRETCH

MVP (must have for submission)

- Single cell: type a description → generates image → renders as particle cloud
- Dark, minimal UI
- Firebase hosted
- Works in Chrome

Stretch (if time allows)

- Grid navigation between multiple cells
- Voice input
- Depth estimation for true 3D displacement
- Ambient audio
- Save/share generated scenes

API KEY SECURITY SOLUTION

Problem: Can't put Gemini API key in client-side JavaScript.

Solution: Firebase Cloud Function as a proxy.

```
// functions/src/index.ts
import * as functions from 'firebase-functions';
import { GoogleGenerativeAI } from '@google/generative-ai';

const genAI = new GoogleGenerativeAI(process.env.GEMINI_API_K
EY!);

export const generateMemory = functions.https.onCall(async (d
ata) => {
  const model = genAI.getGenerativeModel({ model: 'gemini-2.5
-flash-image' });
  const result = await model.generateContent(data.prompt);
  // extract and return base64 image
  return { image: result.response... };
});
```

Store the API key in Firebase environment config:

`firebase functions:config:set gemini.key="YOUR_KEY"`

This keeps the key server-side while the front-end calls the function.

LIVE URL FORMAT

`materealized-web.web.app` or `materealized.web.app` (Firebase provides free .web.app subdomain)